

The transition from front-end development (React and styling) to back-end architecture over the past few weeks has been a rigorous and occasionally frustrating learning process. While I felt increasingly comfortable manipulating data that already existed in the browser, moving to Node.js, Express, and PostgreSQL introduced a completely different mental model. I had to stop thinking about what the user sees and start thinking about how data is securely received, processed, stored, and returned. I used AI as my tutor, specifically I used anti-gravity IDE so it can see my code work and check it, and give me clues on what I did right and wrong.

One of my earliest and most significant struggles was understanding how data moves between the client and the server. I found myself repeatedly confused about when to use ``req.params``, ``req.query``, and ``req.body`` in my Express routes. Testing APIs initially felt smooth; my POST requests would work flawlessly in Postman because I was manually configuring the JSON body. However, things would break as soon as I tried calling that exact same endpoint from my React form. I eventually realized that my frontend ``fetch`` call was missing the ``Content-Type: application/json`` header. Because of this, my Express server (despite having ``app.use(express.json())``) couldn't parse the incoming data, leaving ``req.body`` undefined and causing server crashes whenever I attempted to call string methods like ``.trim()`` on missing fields. This taught me the critical importance of defensive programming and always validating incoming data before acting on it.

Integrating PostgreSQL required another major shift in logic. SQL does not behave like JavaScript array methods. A specific bug that cost me a lot of time involved filtering out null values in a database query. I initially wrote ``WHERE grade_score = NULL``, which is a common pitfall because, in SQL, ``NULL`` is not equal to anything, not even itself. It took time and debugging to realize that ``IS NULL`` is the only mathematically valid way to check for emptiness in a relational database. Additionally, learning to append ``RETURNING *`` to my ``INSERT`` and ``UPDATE`` queries was a breakthrough; without it, I couldn't send the newly created or modified row back to my frontend to update the React state.

The asynchronous nature of database interactions also caught me off guard. Coming from synchronous JavaScript operations, I frequently forgot to place an ``await`` before my ``pool.query()`` calls. This resulted in bizarre race conditions where my Express route would attempt to send a response back to the client before the data was actually written to the database, returning unresolved Promises instead of the expected row data.

I want to be fully transparent about my use of AI during the coding. I have utilized AI extensively, not to write my code for me, but as an interactive tutor to help me decode the vague error messages I encountered. For example, when my ``GROUP BY`` SQL statements were returning fragmented counts, rather than just asking for the right query, I asked the AI to explain **why** my logic was failing mathematically. I also used AI to generate "advanced" practice scenarios with edge cases, like handling missing default dates in an ``INSERT`` to test my understanding. This iterative questioning process has significantly improved my ability to independently read stack traces and differentiate between a client-side network error and a server-side logic crash.

All of these backend struggles and breakthroughs have directly prepared me for my final project, LiturgyGen. In my initial proposal, I planned a system that needs an Express backend to fetch Mass readings based on liturgical dates and a PostgreSQL database to manage custom Prayers of the Faithful (POTF) templates. Previously, the idea of connecting a React dashboard to a database that stores these templates seemed overwhelming. Now, I understand exactly how I will structure my API routes to fetch POTF templates and how crucial defensive programming (like validating ``req.body``) will be when Campus Ministry staff submit new templates via a ``POST`` request. Furthermore, knowing that I must explicitly ``await`` asynchronous database calls and use ``RETURNING *`` ensures that when the staff adds a new template, the React frontend will immediately receive and display the updated data.

In summary, this half of the course has shown me that building a functional backend requires attention to detail. I feel much more confident entering the midterm now that I understand the distinct responsibilities of the frontend, the routing layer, and the database.